

Estructuras de Datos - Repaso

1 Uso de Comparable

La interfaz Comparable<T> permite definir un orden natural entre objetos de una clase. Se implementa el método compareTo(T o) y se usa, por ejemplo, en Collections.sort(lista).

```
1 public class Estudiante implements Comparable<Estudiante> {
2     private String nombre;
3     private int edad;
4
5     // Constructor, getters...
6
7     @Override
8     public int compareTo(Estudiante otro) {
9         return Integer.compare(this.edad, otro.edad); // Ascendente
10    }
11 }
```

2 Ordenamiento por Comparator (ejemplos)

1. Orden descendente por edad

```
1 lista.sort((e1, e2) -> Integer.compare(e2.getEdad(), e1.getEdad()));
```

2. Orden por múltiples criterios: edad y luego nombre

```
1 lista.sort(Comparator
2     .comparingInt(Estudiante::getEdad)
3     .thenComparing(Estudiante::getNombre));
```

3. Ejemplo completo con todos los criterios

```
1 public static void main(String[] args) {
2     List<Estudiante> lista = new ArrayList<>();
3     lista.add(new Estudiante("Ana", 22));
4     lista.add(new Estudiante("Luis", 22));
5     lista.add(new Estudiante("Carla", 20));
6
7     // Ascendente por edad y nombre
```

```

8      lista.sort(Comparator
9          .comparingInt(Estudiante::getEdad)
10         .thenComparing(Estudiante::getNombre));
11
12     for (Estudiante e : lista) {
13         System.out.println(e);
14     }
15
16     // Descendente por edad
17     lista.sort((e1, e2) -> Integer.compare(e2.getEdad(), e1.getEdad()));
18
19     for (Estudiante e : lista) {
20         System.out.println(e);
21     }
22 }

```

3 Uso genérico de Comparator con ¡T!

Comparator permite escribir funciones genéricas para ordenar cualquier lista.

```

1 public class Ordenador {
2
3     // Ordenar genéricamente por un campo comparable
4     public static <T> void ordenarPorCampo(List<T> lista, Comparator<T> comparador)
5     {
6         Collections.sort(lista, comparador);
7     }
8
9     public static void main(String[] args) {
10         List<Estudiante> lista = new ArrayList<>();
11         lista.add(new Estudiante("Ana", 22));
12         lista.add(new Estudiante("Luis", 20));
13
14         // Usar función genérica para ordenar por nombre
15         ordenarPorCampo(lista, Comparator.comparing(Estudiante::getNombre));
16
17         for (Estudiante e : lista) {
18             System.out.println(e);
19         }
20     }
21 }

```

Conclusión

- Usa Comparable para definir un único orden natural.
- Usa Comparator para múltiples criterios y ordenamientos personalizados.
- Puedes generalizar lógica con <T> para reutilizar métodos de ordenamiento.

Ejercicio 2: Uso de Comodines para Encontrar el Elemento Menor

Descripción: Implementa un método genérico que reciba una lista de objetos que extiendan `Comparable<T>` y devuelva el **elemento menor** de la lista.

Instrucciones:

1. Implementa el método `encontrarMenor(List<? extends Comparable<T>> lista)` en una clase llamada `Utilidades`.
2. Crea listas de `String` y `Double`, y prueba el método con ambas para encontrar el elemento menor.

Ejemplo de uso:

```
1 List<Double> listaDoubles = Arrays.asList(2.5, 1.0, 3.7);
2 System.out.println(Utilidades.encontrarMenor(listaDoubles));
3 // Salida: 1.0
4
5 List<String> listaStrings = Arrays.asList("z", "x", "a");
6 System.out.println(Utilidades.encontrarMenor(listaStrings));
7 // Salida: a
```

Pista de implementación:

```
1 public class Utilidades {
2     public static <T extends Comparable<T>> T encontrarMenor(List<? extends T> lista
3     ) {
4         if (lista == null || lista.isEmpty()) {
5             throw new IllegalArgumentException("La lista no puede ser nula o vac a.
6             ");
7         }
8         T menor = lista.get(0);
9         for (T elemento : lista) {
10             if (elemento.compareTo(menor) < 0) {
11                 menor = elemento;
12             }
13         }
14         return menor;
15     }
16 }
```